LAPORAN PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

MODUL 6 – Inheritance dan Polymorphism



RAYMUNDUS ARIEL ABAS

103132430021

S1SD-05-01

PROGRAM STUDI S-1 SAINS DATA

FAKULTAS DIREKTORAT KAMPUS PURWOKERTO

TELKOM UNIVERSITY

2026

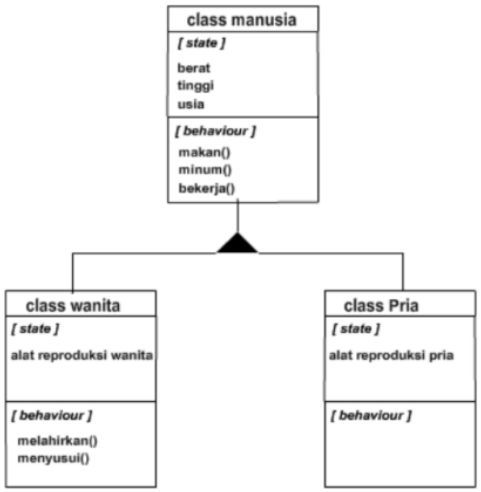
A. Dasar Teori

1. Pewarisan (*Inheritance*)

Inheritance (pewarisan) merupakan salah satu pilar utama dalam paradigma *Object-Oriented Programming* (OOP) yang memungkinkan pembentukan hierarki kelas melalui penurunan sifat dari entitas umum menuju entitas yang lebih spesifik. Dalam bahasa pemrograman Java, mekanisme ini diimplementasikan agar sebuah kelas dapat memperluas fungsionalitas kelas lain secara terstruktur tanpa perlu menulis ulang kode dasar yang sama.

Secara konseptual, struktur pewarisan terbagi menjadi dua entitas utama:

- Kelas Induk (*Superclass* / *Base Class*): Kelas yang bertindak sebagai entitas dasar dan mendefinisikan karakteristik umum yang akan diturunkan ke kelas lain.
- Kelas Turunan (*Subclass* / *Derived Class*): Kelas spesifik yang mewarisi atribut (*state*) dan perilaku (*behaviour*) dari *superclass*, sekaligus dapat menambahkan atribut maupun fungsionalitas unik miliknya sendiri.



Berdasarkan diagram kelas di atas, kelas Manusia berperan sebagai *superclass* yang memiliki atribut umum seperti berat, tinggi, dan usia, serta perilaku umum seperti makan, minum, dan bekerja. Kelas Pria dan Wanita bertindak sebagai *subclass* yang secara otomatis mewarisi seluruh karakteristik dasar kelas Manusia, namun masing-masing memiliki atribut dan fungsionalitas biologis khusus yang membedakannya satu sama lain.

2. Super Keyword

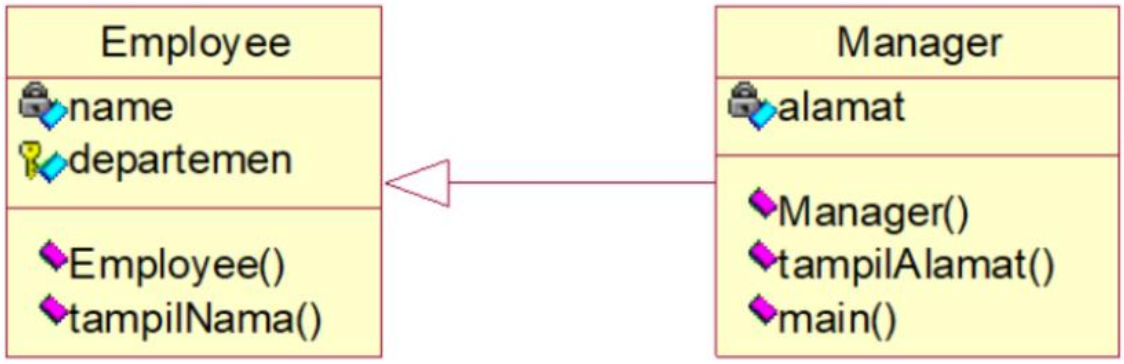
Dalam bahasa pemrograman Java, kata kunci super berperan sebagai variabel referensi yang merujuk langsung ke kelas induk (*superclass*). Kata kunci ini digunakan

oleh kelas turunan (*subclass*) untuk mengakses elemen internal milik *superclass*, baik berupa variabel (atribut), konstruktor, maupun metode (*method*).

Secara umum, pemanfaatan kata kunci super mencakup:

- Pemanggilan Konstruktor Induk: Menginisialisasi nilai atribut bawaan dari *superclass* langsung dari dalam konstruktor *subclass* melalui sintaks super().
- Mengakses Atribut Induk: Mengambil nilai variabel milik *superclass* apabila terjadi kesamaan nama variabel (*variable shadowing*) di dalam *subclass*.
- Memanggil Metode yang Ditimpa (*Overridden Method*): Menjalankan implementasi asli dari sebuah metode milik *superclass* yang telah diubah atau ditimpa kodenya di dalam *subclass*.

Contoh:



3. *Method Overloading*

Method overloading merupakan fitur dalam *Object-Oriented Programming* (OOP) yang memungkinkan sebuah kelas mendefinisikan dua atau lebih metode dengan identitas (nama) yang sama. Agar dapat dikenali oleh kompilator Java tanpa menimbulkan ambiguitas, setiap metode wajib memiliki parameter masukan (*method signature*) yang unik dan berbeda satu sama lain.

Pembedaan parameter pada *overloading* dapat ditentukan melalui aspek berikut:

- Jumlah Parameter: Menyediakan jumlah argumen masukan yang berbeda pada setiap metode dengan nama yang sama.
- Tipe Data Parameter: Menggunakan tipe data yang berlainan untuk urutan argumen tertentu (misalnya int, double, atau objek).
- Urutan Tipe Data Parameter: Menyusun kombinasi urutan tipe data yang berbeda meskipun jenis parameternya sama.

Penerapan *method overloading* meningkatkan fleksibilitas dan keterbacaan kode (*code readability*), karena pengembang dapat menggunakan satu nama metode intuitif untuk menangani berbagai format data masukan tanpa perlu menciptakan nama metode baru yang redundan.

4. *Method Overriding*

Method overriding merupakan mekanisme dalam *Object-Oriented Programming* (OOP) di mana sebuah kelas turunan (*subclass*) mendefinisikan ulang metode yang telah dideklarasikan sebelumnya pada kelas induk (*superclass*). Metode penimpa ini wajib memiliki nama, tipe nilai kembalian (*return type*), serta daftar parameter (*signature*) yang identik dengan metode milik *superclass*.

Karakteristik dan implikasi dari penerapan *method overriding* meliputi:

- Modifikasi Perilaku: Memungkinkan *subclass* memberikan implementasi spesifik yang relevan dengan kebutuhannya tanpa merusak struktur kontrak metode yang telah ditentukan oleh *superclass*.

- Prioritas Eksekusi: Saat metode tersebut dipanggil melalui instansi (*object*) milik *subclass*, program akan memprioritaskan dan mengeksekusi logika yang tertulis di dalam *subclass*, mengabaikan implementasi bawaan dari *superclass*.

- Polimorfisme Dinamis: Menjadi fondasi terbentuknya *runtime polymorphism* (ikatan dinamis), di mana Java menentukan versi metode yang dijalankan berdasarkan tipe objek aktual saat program berjalan.

- Anotasi @Override: Praktik penulisan kode Java yang dianjurkan untuk menyertakan anotasi @Override guna memastikan validasi kompilator bahwa metode tersebut benar-benar menimpa metode milik kelas induk.

B. Guided

1. Guided1

a. A.java

```
package raymundusariel.guided.guided1;


public class A {


    int x;

    int y;


    void tampilXY() {

        System.out.println("Nilai x: " + x + " nilai y: " + y);

    }

}
```

b. B.java

```
package raymundusariel.guided.guided1;


public class B extends A {

    int z;

    int hasil;


    void jumlahXYZ() {

        hasil = x+y+z;


        System.out.println("jumlah (x + y + z) = " + hasil);
```

```
    }
}
```

c. Demo/Main.java

```
package raymundusariel.guided.guided1;
public class Demo {
    public static void main(String[] args) {
        A superclass = new A();
        B subclass = new B();

        superclass.x = 3;
        superclass.y = 4;

        System.out.println("Superclass: ");
        superclass.tampilXY();


        // member superclass dapat diakses dari subclassnya
        subclass.x = 1;
        subclass.y = 2;
        subclass.z = 5;

        System.out.println("Subclass: ");
        subclass.jumlahXYZ();
```

```
        }
    }
```

Output:

```
--- exec:3.6.3:exec (default-cli) @ guided ---
Superclass:
Nilai x: 3 nilai y: 4
Subclass:
jumlah (x + y + z) = 8
------------------------------------------------
```

2. Guided2

a. Employee.java

```
package raymundusariel.guided.guided2;

public class Employee {
    private String name;
    String departemen;

    public Employee(String nama) {
        this.name = nama;
    }

    public void tampilNama() {
        System.out.println("nama : " + name);
    }
}
```

b. Manager.java

```
package raymundusariel.guided.guided2;
```

```
public class Manager extends Employee {
    private String alamat;

    public Manager(String nama, String alamat) {
        /* memanggil konstruktor employee */
        super(nama);
        this.alamat = alamat;
    }

    public void tampilInformasi() {
        super.tampilNama();

        /* menampilkan variabel alamat dan departemen */
        System.out.println("alamat : " + alamat);
        System.out.println("departemen : " + super.departemen);
    }
}
```

c. DemoManager.java

```
package raymundusariel.guided.guided2;
public class DemoManager {
    public static void main(String[] args) {
        /* membuat objek */
        Manager adi = new Manager("Adi", "Sukabirus");
```

```
        adi.departemen = "SDM";
        adi.tampilInformasi();
    }
}
```

Output:

```
--- exec:3.6.3:exec (default-cli) @ guided ---
nama : Adi
alamat : Sukabirus
departemen : SDM
------------------------------------------------
```

3. Guided3

a. Lagu.java

```
package raymundusariel.guided.guided3;

public class Lagu {
    String pencipta;
    String judul;

    void IsiParam(String param1) {
        judul = param1;
        pencipta = "Tidak dikenal";
    }

    void IsiParam(String param1, String param2) {
        judul = param1;
        pencipta = param2;
    }
```

```
    void Tampilkan() {
        System.out.println("Judul : " + judul + ", pencipta : " + pencipta);
    }
}
```

b. DemoOver.java

```
package raymundusariel.guided.guided3;


public class DemoOver2 {
    public static void main(String[] args) {
        Lagu d, e;
        d = new Lagu();
        e = new Lagu();


        d.IsiParam("Lagu 1");
        e.IsiParam("Lagu 2", "aku");


        d.Tampilkan();
        e.Tampilkan();
    }
}
```

Output:

```
--- exec:3.6.3:exec (default-cli) @ guided ---
Judul : Lagu 1, pencipta : Tidak dikenal
Judul : Lagu 2, pencipta : aku
------------------------------------------------
```

4. Guided4

a. A.java

```
package raymundusariel.guided.guided4;
public class A {

    public void tampilkan() {
        System.out.println("Method milik class A dipanggil...");
    }
}
```

b. B.java

```
package raymundusariel.guided.guided4;
public class B extends A {

    public void tampilkan() {
        super.tampilkan(); // Method milik superclas dipanggil
        System.out.println("Method milik class B dipanggil...");
    }
}
```

c. DemoInheritance

```
package raymundusariel.guided.guided4;
public class DemoInheritance {

    public static void main(String[] args) {
```

```
        B overidB = new B();
        overidB.tampilkan();
    }
}
```

Output:

```
--- exec:3.6.3:exec (default-cli) @ guided ---
Method milik class A dipanggil...
Method milik class B dipanggil...
```

C. Unguided

1. Soal Unguided 1

Buatlah program Java untuk menghitung dan membandingkan biaya komputasi tiga jenis model machine learning, yaitu model regresi, model klasifikasi, dan model klasifikasi deep learning.

Ketentuan program:

- Buat kelas induk ModelML dengan atribut nama model dan jumlah data latih, serta method untuk menampilkan informasi dan menghitung biaya komputasi menggunakan konsep overloading.
- Buat kelas ModelRegresi yang mewarisi ModelML dengan atribut jumlah fitur. Terapkan overriding pada method informasi dan perhitungan biaya, serta tambahkan overloading untuk menghitung biaya berdasarkan epoch dan ukuran batch.
- Buat kelas ModelKlasifikasi yang mewarisi ModelML dengan atribut jumlah kelas. Terapkan overriding pada method informasi dan perhitungan biaya.
- Buat kelas ModelDeepLearning yang mewarisi ModelKlasifikasi dengan atribut jumlah layer. Terapkan overriding pada method informasi dan perhitungan biaya.
- Buat kelas Main untuk membuat objek Regresi Linier (500 data latih, 4 fitur), Decision Tree (800 data latih, 3 kelas), dan Neural Network (1.000 data latih, 5 kelas, 3 layer). Simpan ketiganya dalam array bertipe ModelML, tampilkan informasi dan biaya setiap model, hitung total biaya, serta tampilkan biaya untuk 10 epoch dan biaya model regresi dengan 10 epoch dan ukuran batch 50.

Seluruh atribut harus bersifat private. Gunakan constructor, inheritance, method overriding, method overloading, dan polymorphism sesuai ketentuan soal.

Jawaban:

ModelML.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt
to change this license
 */

package raymundusariel.unguided;

/**
 *
 * @author ACER
 */
public class ModelML {
    private String nama;
    private int dataLatih;

    public ModelML(String nama, int dataLatih) {
        this.nama = nama;
        this.dataLatih = dataLatih;
    }

    public int getDataLatih() {
        return this.dataLatih;
    }

    public void tampilkanInfo() {
        System.out.println("Model: " + this.nama
                + " | Data latih: " + this.dataLatih);
    }

    public int hitungBiaya() {
        return this.dataLatih * 2;
    }

    public int hitungBiaya(int epoch) {
        return this.dataLatih * epoch;
    }
}
```

ModelRegresi.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt
to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit
this template
 */
package raymundusariel.unguided;

/**
 *
 * @author ACER
 */
public class ModelRegresi extends ModelML {
    private int jumlahFitur;

    public ModelRegresi(String nama, int dataLatih, int jumlahFitur) {
        super(nama, dataLatih);
        this.jumlahFitur = jumlahFitur;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah fitur: " + this.jumlahFitur);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (this.jumlahFitur * 10);
    }

    public int hitungBiaya(int epoch, int ukuranBatch) {
        return (getDataLatih() * epoch) / ukuranBatch;
    }
}
```

ModelKlasifikasi.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package raymundusariel.unguided;

/**
 *
 * @author ACER
 */
public class ModelKlasifikasi extends ModelML {
    private int jumlahKelas;

    public ModelKlasifikasi(String nama, int dataLatih, int jumlahKelas) {
        super(nama, dataLatih);
        this.jumlahKelas = jumlahKelas;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah kelas: " + this.jumlahKelas);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() * this.jumlahKelas;
    }
}
```

ModelDeepLearning.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package raymundusariel.unguided;

/**
 *
 * @author ACER
 */
public class ModelDeepLearning extends ModelKlasifikasi {
    private int jumlahLayer;

    public ModelDeepLearning(String nama, int dataLatih,
                             int jumlahKelas, int jumlahLayer) {
        super(nama, dataLatih, jumlahKelas);
        this.jumlahLayer = jumlahLayer;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah layer: " + this.jumlahLayer);
```

```
    }


    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (this.jumlahLayer * 200);
    }
}
```

Main.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package raymundusariel.unguided;


/**
 *
 * @author ACER
 */
public class Main {
    public static void main(String[] args) {
        ModelML[] daftarModel = {
            new ModelRegresi("Regresi Linier", 500, 4),
            new ModelKlasifikasi("Decision Tree", 800, 3),
            new ModelDeepLearning("Neural Network", 1000, 5, 3)
        };

        int totalBiaya = 0;

        for (ModelML model : daftarModel) {
            model.tampilkanInfo();
```

```
            int biaya = model.hitungBiaya();
            System.out.println("Biaya komputasi: " + biaya);
            System.out.println("---");

            totalBiaya += biaya;
        }

        System.out.println("Total biaya: " + totalBiaya);

        System.out.println("Biaya 10 epoch (Neural Network): "
                + daftarModel[2].hitungBiaya(10));

        System.out.println("Biaya 10 epoch (Regresi Linier): "
                + daftarModel[0].hitungBiaya(10));

        ModelRegresi regresi = (ModelRegresi) daftarModel[0];

        System.out.println("Biaya 10 epoch batch 50 (Regresi Linier): "
                + regresi.hitungBiaya(10, 50));
    }
}
```

Penjelasan Program

Program ini menerapkan konsep Pemrograman Berorientasi Objek (PBO), yaitu enkapsulasi, inheritance, polymorphism, method overloading, dan method overriding.

1. Enkapsulasi

Seluruh atribut, seperti nama, dataLatih, jumlahFitur, jumlahKelas, dan jumlahLayer, menggunakan access modifier private untuk membatasi akses langsung dari luar kelas. Atribut dataLatih dapat dibaca melalui getter getDataLatih() yang disediakan oleh kelas ModelML.

2. Inheritance (Pewarisan)

Kelas ModelRegresi dan ModelKlasifikasi mewarisi kelas ModelML, sedangkan ModelDeepLearning mewarisi kelas ModelKlasifikasi. Hubungan ini membentuk pewarisan bertingkat. Setiap constructor kelas turunan menggunakan super(...) untuk memanggil constructor kelas induknya.

3. Method Overloading

Overloading diterapkan pada method hitungBiaya() yang memiliki nama sama dengan parameter berbeda. Kelas ModelML menyediakan versi tanpa parameter dan versi dengan satu parameter epoch. Kelas ModelRegresi menambahkan versi dengan dua parameter, yaitu epoch dan ukuranBatch.

4. Method Overriding

Overriding diterapkan pada method tampilkanInfo() dan hitungBiaya() di kelas-kelas turunan. Method tampilkanInfo() memanggil super.tampilkanInfo() untuk menampilkan informasi dasar, kemudian menambahkan atribut khusus setiap model.

Perhitungan biaya juga disesuaikan dengan jenis model:

- ModelRegresi: biaya dasar ditambah jumlah fitur dikalikan 10.
- ModelKlasifikasi: biaya dasar dikalikan jumlah kelas.
- ModelDeepLearning: biaya klasifikasi ditambah jumlah layer dikalikan 200.

5. Polimorfisme dan Downcasting

Polimorfisme diterapkan dengan menyimpan objek ModelRegresi, ModelKlasifikasi, dan ModelDeepLearning ke dalam array bertipe ModelML[]. Ketika method hitungBiaya() dipanggil melalui elemen array, Java menjalankan implementasi yang sesuai dengan jenis objek sebenarnya.

Downcasting dilakukan untuk mengakses method hitungBiaya(int epoch, int ukuranBatch) yang hanya tersedia pada kelas ModelRegresi. Objek regresi diambil dari array dan dikonversi ke tipe ModelRegresi agar method tersebut dapat dipanggil.

D. Kesimpulan

Praktikum Modul 6 menunjukkan bahwa mekanisme *Inheritance* memungkinkan pewarisan atribut dan method secara hierarkis, baik melalui pewarisan langsung maupun bertingkat, sehingga kode dapat digunakan kembali tanpa perlu menulis ulang seluruh implementasi.

Penerapan *Method Overriding* dan *Polimorfisme* memungkinkan berbagai jenis model diproses secara seragam melalui array bertipe kelas induk, dengan tetap menjalankan perhitungan biaya dan menampilkan informasi sesuai jenis objeknya. Selain itu, *Method Overloading* memberikan fleksibilitas dalam penggunaan method dengan nama yang sama tetapi parameter berbeda, sedangkan *Downcasting* memungkinkan akses ke method khusus kelas turunan yang tidak tersedia pada kelas induk.